

Implementacija genetskog algoritma za višestruko poravnanje sekvenci

Multiple Sequence Alignment (MSA)

Mileta Jovanović 32/2020

Problem MSA

Multiple Sequence Alignment predstavlja proces poravnanja više bioloških sekvenci (npr. DNK, RNK ili proteina) sa ciljem da se identifikuju mesta koja su evolutivno ili funkcionalno srodna između sekvenci.

NP-kompletan problem - nema efikasnog algoritma za optimalno rešenje

Broj mogućih poravnanja raste **eksponencijalno** sa brojem i dužinom sekvenci

Potrebni su **heuristički pristupi** za praktične primene

Velika primena u bioinformatici

Primer poravnanja

Originalne sekvence:

1. PAWHE

2. HEAWY

3. AWHE

Ideja je da se sve sekvence predstave u **tabelarnom** obliku jednake dužine, tako što se po potrebi ubacuju praznine (gap karakter „-“) kako bi se slični ili identični simboli poravnali u istim kolonama.

Poravnanje:

1. P-AWHE

2. HEAW-Y

3. -AW-HE

Na taj način se maksimizuje ukupan skor poravnanja, odnosno mera sličnosti između svih sekvenci.

Fitness funkcija - SP skor

Sum-of-Pairs (SP) skor meri kvalitet poravnanja sabiranjem skorova svih mogućih parova sekvenci.

$$SP(A) = \sum_{i < j} \sum_k score(A[i][k], A[j][k])$$

gde je:

A – matrica poravnanja,

i, j – indeksi sekvenci u poravnanju,

k – indeks pozicije (kolone) u poravnanju.

U našoj implementaciji SP skora koristimo:

- **BLOSUM62 supstitucionu matricu** - ocenjuje sličnost između parova sekvenci
- **Affine gap penalty** - uvodi se kazna za praznine
 - gap open penalty (−10)
 - gap extend penalty (−1).

$$\text{Gap}(L) = \text{gap_open} + (L - 1) \times \text{gap_extend}$$

Algoritam grube sile

```
def generate_all_gap_variants(seq, max_gaps):
    from itertools import combinations

    variants = []

    for num_gaps in range(max_gaps + 1):
        if num_gaps == 0:
            variants.append(seq)
            continue

        total_len = len(seq) + num_gaps

        for gap_positions in combinations(range(total_len), num_gaps):
            result = []
            seq_idx = 0
            gap_set = set(gap_positions)

            for i in range(total_len):
                if i in gap_set:
                    result.append('-')
                else:
                    result.append(seq[seq_idx])
                    seq_idx += 1

            variants.append(''.join(result))

    return variants
```

```
def brute_force_msa(sequences, max_gaps=3):
    all_variants = []
    for seq in sequences:
        variants = generate_all_gap_variants(seq, max_gaps)
        all_variants.append(variants)

    best_alignment = None
    best_score = float('-inf')

    for combo in product(*all_variants):
        # print(combo)
        max_len = max(len(s) for s in combo)
        alignment = [s + '-' * (max_len - len(s)) for s in combo]

        score = calculate_sp_score(alignment)

        if score > best_score:
            best_score = score
            best_alignment = alignment

    return best_alignment, best_score
```

Reprezentacija jedinke i inicijalna populacija

Struktura jedinke:

- `original_sequences` - liste originalnih
- `max_gaps` - maksimalan broj praznina po sekvenci
- `gap_positions` - lista pozicija praznina za svaku sekvencu
- `code` - finalno poravnanje (sekvence sa umetnutim prazninama)
- `fitness`: SP skor poravnanja

Inicijalna populacija:

- Svaka jedinka generisana je pseudonasumično
 - Nasumično se bira broj gapova iz intervala $[0, \text{max_gaps}]$
 - Za svaku sekvencu u poravnanju se biraju nasumične lokacije za praznine

Selekcija

Turnirska - bira najbolju od n nasumičnih jedinki izabranih iz populacije

Rulet - verovatnoćom proporcionalnom fitnessu vraća jedinku

Rang - bazirana na rangu umesto fitness vrednosti, eliminiše uticaj ekstremnih fitnessa

```
def roulette_selection(population, _):  
    # Shift values to positive  
    min_fitness = min(ind.fitness for ind in population)  
    if min_fitness < 0:  
        adjusted_fitness = [ind.fitness - min_fitness + 1 for ind in population]  
    else:  
        adjusted_fitness = [ind.fitness for ind in population]  
  
    total_fitness = sum(adjusted_fitness)  
    if total_fitness <= 0:  
        return random.choice(population)  
  
    # Roulette wheel  
    pick = random.uniform(0, total_fitness)  
    current = 0  
    for ind, fitness in zip(population, adjusted_fitness):  
        current += fitness  
        if current > pick:  
            return ind  
    return population[-1]
```

```
def tournament_selection(population, tournament_size):  
    k = min(len(population), tournament_size)  
    participants = random.sample(population, k)  
    return max(participants, key=lambda x: x.fitness)
```

Ukrštanje

Single point

- Bira se po jedna tačka iz gap_positions oba roditelja
- U slučaju da jedan od roditelja nema praznine, radi se zamena gap_positions za decu

```
def crossover(parent1, parent2, child1, child2):
    child1.gap_positions = []
    child2.gap_positions = []

    for i in range(len(parent1.gap_positions)):
        gaps1 = parent1.gap_positions[i]
        gaps2 = parent2.gap_positions[i]

        # Only do crossover if both parents have gaps
        if len(gaps1) > 0 and len(gaps2) > 0:
            min_len = min(len(gaps1), len(gaps2))
            split = random.randint(0, min_len)

            child1.gap_positions.append(sorted(gaps1[:split] + gaps2[split:]))
            child2.gap_positions.append(sorted(gaps2[:split] + gaps1[split:]))
        else:
            # If we one of the parents doesn't have gaps, we switch them for children
            child1.gap_positions.append(gaps2[:])
            child2.gap_positions.append(gaps1[:])

    child1.code = child1.build_alignment()
    child2.code = child2.build_alignment()
```


Mutacija

Može biti sačinjena od narednih funkcija

Add - dodaje novu prazninu

Remove - uklanja prazninu

Move - pomera postojeću prazninu

Implementirane su dve varijante koje:

1. Add_Remove_Move
2. Add_Move

```
def mutation_add_remove_move(child, mutation_prob):  
    # Go through each sequence of alignment  
    for i in range(len(child.gap_positions)):  
  
        if random.random() < mutation_prob:  
            seq_len = len(child.original_sequences[i])  
            current_gaps = child.gap_positions[i]  
  
            # Randomly choose to insert, remove, or move a gap  
            mutation_type = random.choice(['add', 'remove', 'move'])  
  
            # Add a gap at randomly chosen position  
            if mutation_type == 'add' or len(current_gaps) == 0:  
                if len(current_gaps) < child.max_gaps:  
                    total_len = seq_len + len(current_gaps)  
                    available = [p for p in range(total_len + 1) if p not in current_gaps]  
                    if available:  
                        new_gap = random.choice(available)  
                        current_gaps.append(new_gap)  
                        child.gap_positions[i] = sorted(current_gaps)  
            # Remove a random gap  
            elif mutation_type == 'remove' and len(current_gaps) > 0:  
                current_gaps.pop(random.randint(0, len(current_gaps) - 1))  
  
            # Switch position of a gap  
            elif mutation_type == 'move' and len(current_gaps) > 0:  
                old_gap_idx = random.randint(0, len(current_gaps) - 1)  
                current_gaps.pop(old_gap_idx)  
                total_len = seq_len + len(current_gaps)  
                available = [p for p in range(total_len + 1) if p not in current_gaps]  
                if available:  
                    new_gap = random.choice(available)  
                    current_gaps.append(new_gap)  
                    child.gap_positions[i] = sorted(current_gaps)  
  
    # Rebuild alignment from modified gap positions  
    child.code = child.build_alignment()
```

Parametri GA

Konfiguracija	Selekcija	Mutacija	Pop.	Gen.	Tour. size	Mut. Prob.	Elitism
Tour AddRemMove	Turnir	Add Remove Move	100	50	10	0.10	10
Tour AddRem	Turnir	Add Remove	100	50	10	0.12	10
Roul AddRemMove	Rulet	Add Remove Move	100	50	-	0.10	10
Rank AddRemMove	Rank	Add Remove Move	100	50	-	0.10	10
Tour Large	Turnir	Add Remove Move	150	75	20	0.08	15

ClustalW i BAliBase test skup

ClustalW je jedan od najpopularnijih progresivnih algoritama za višestruko poravnanje sekvenci (MSA).

Algoritam gradi poravnanje postepeno, počevši od najbližih sekvenci i postupno dodajući udaljenije sekvence.

Za evaluaciju performansi koristili smo **BAliBASE** (Benchmark Alignment dataBASE), jedan od najpoznatijih benchmark skupova za MSA. Konkretno podskup testova **RV11**.

ClustalW je pokrenut sa parametrima prilagođenim implementaciji fntes funkcije

Tip: PROTEIN

Matrica: BLOSUM62

Gap open penalty: 10

Gap extend penalty: 1

Rezultati

ClustalW (crvena linija) konzistentno postiže najbolje skorove, dok GA konfiguracije prate sličan trend sa nižim vrednostima.

Konfiguracija	Avg sc.	Avg time
Tour_AddRemMove	-9184.93	14.368s
Tour_AddRem	-9008.47	14.376s
Roul_AddRemMove	-8989.32	14.347s
Rank_AddRemMove	-9147.68	14.347s
Tour_Large	-8745.72	32.208s
ClustalW	-4856.68	0.15s

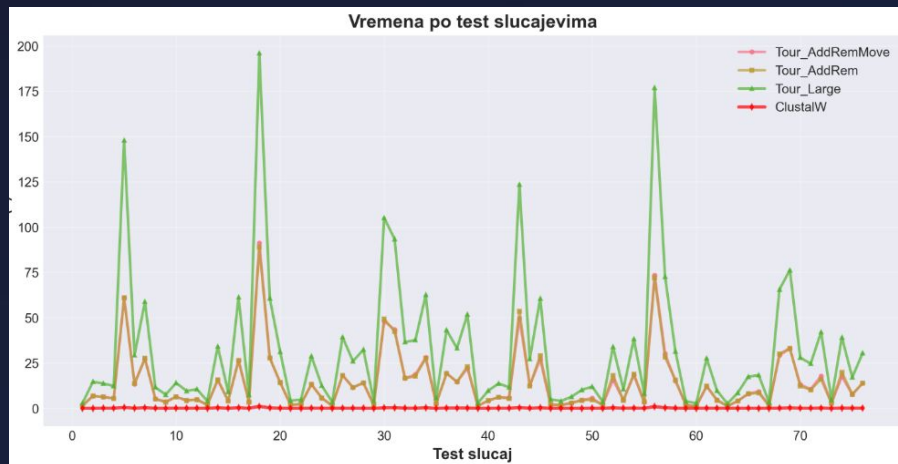
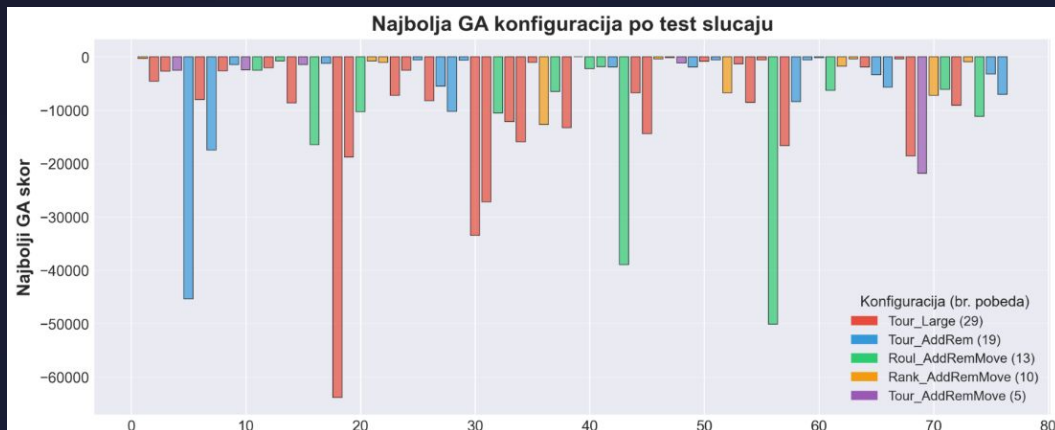


Rezultati

Analiza raspodele najboljih rezultata po test slučajevima pokazuje da se

1. Tour_Large - 29 testova
2. Roul_AddRemMove - 13 testova
3. Rank_AddRemMove - 10 testova
4. Tour_AddRem - 9 testova
5. Tour_AddRemMove - 5 testova

Kod Tour_Large povećana inicijalna populacija i broj generacija dovele su do poboljšanja rezultata, ali na veliki uštrb efikasnosti.



Zaključak

Ključni nalazi:

GA pokazuje obećavajuće rezultate na manjim test slučajevima

ClustalW nadmašuje **GA** u kvalitetu i brzini izvršavanja

Turnir + Add-Remove-Move najbolja GA kombinacija

Prostor pretrage je **ogroman** ($\sim 10^{14}$ mogućnosti)

Potencijalna poboljšanja

- Hibridni pristup: GA + ClustalW inicijalizacijom
- Adaptivni parametri: Dinamičko podešavanje `mutation_prob` i `tournament_size` tokom evolucije